ATTACK & DEFENSE

SELF-PACED PROJECT

CREATED BY: ADAM SNYDER & KYLE QUESTAD
GRAND CANYON UNIVERSITY

Table of Contents

Project Summary (finish at the end)

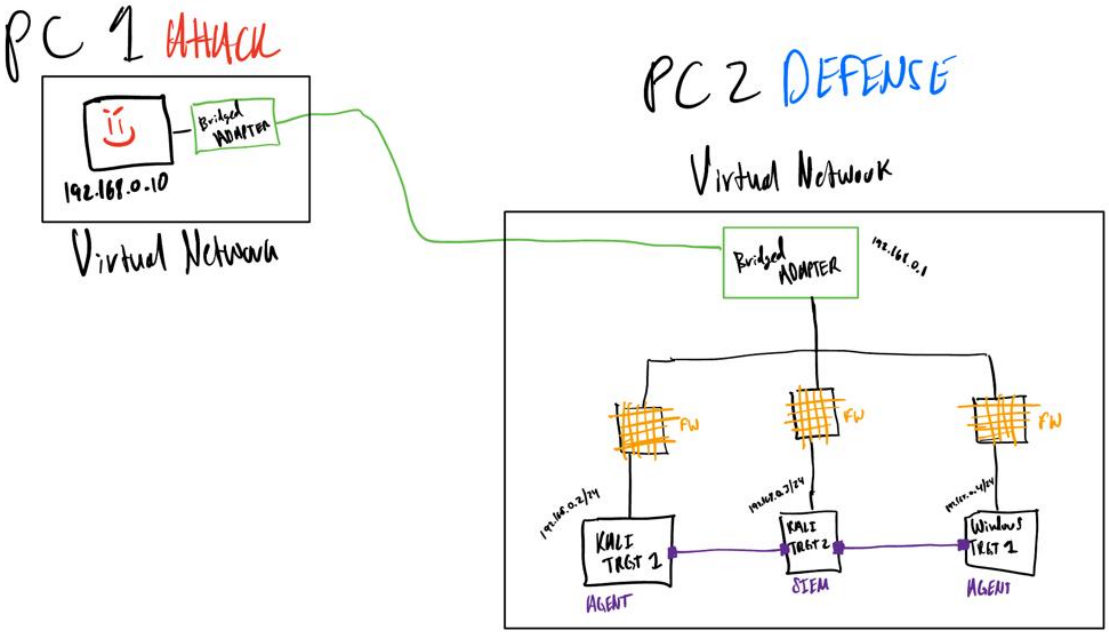
In this project, we designed and implemented a comprehensive virtualized lab environment consisting of eight virtual machines connected through a custom-configured LAN to simulate a realistic enterprise network. The lab incorporated a variety of operating systems, including Linux, Ubuntu, and Windows, to reflect the diversity of systems commonly found in real-world infrastructures. On the defensive side, we deployed multiple blue team tools, including Wazuh as a centralized SIEM for log aggregation, analysis, and alerting, along with Snort and Suricata as network-based intrusion detection systems. To enhance visibility and detection efficiency, we also created and implemented custom Snort rules tailored to our environment, allowing for more meaningful and organized alert data within Wazuh. In parallel, we configured a dedicated Kali Linux virtual machine to act as an attacker, performing a range of offensive techniques such as network scanning, enumeration, and exploitation. These simulated attacks enabled us to observe how malicious activity is captured, logged, and correlated across different security tools, providing valuable insight into incident detection and response workflows. Overall, this project was developed to provide hands-on experience with industry-standard cybersecurity tools, strengthen our understanding of both offensive and defensive operations, and serve as a strong resume-building project that demonstrates practical technical skills and real-world problem-solving abilities.

Planning + Changes Made

3/24/2026

- Make sure to include:
    - IDS/IPS (Snort)
    - AI of some sort
    - 1 Ubuntu, 1 Kali & 1 Window VM (Defensive)
    - Kali for Attacking VM
    - Python Scripting
    - CVSS score
    - Possibly add
        - Soar
        - EDR
        - Splunk
        - wNessus Vulnerability scanning, Red team exploits misconfigured ports
        - IR report
        - TCP dump
        - Security onion
        - NIST framework
        -
- Overall Thoughts:
    - Simulate an attack on a network of Kali VM's
    - Have the VM's alert a SIEM (Wazuh)
    - Have AI perform Triaging and escalate to the IR team
    - Go through a set of procedures simulating a real incident response team (Framework: Lockheed Kill Chain)

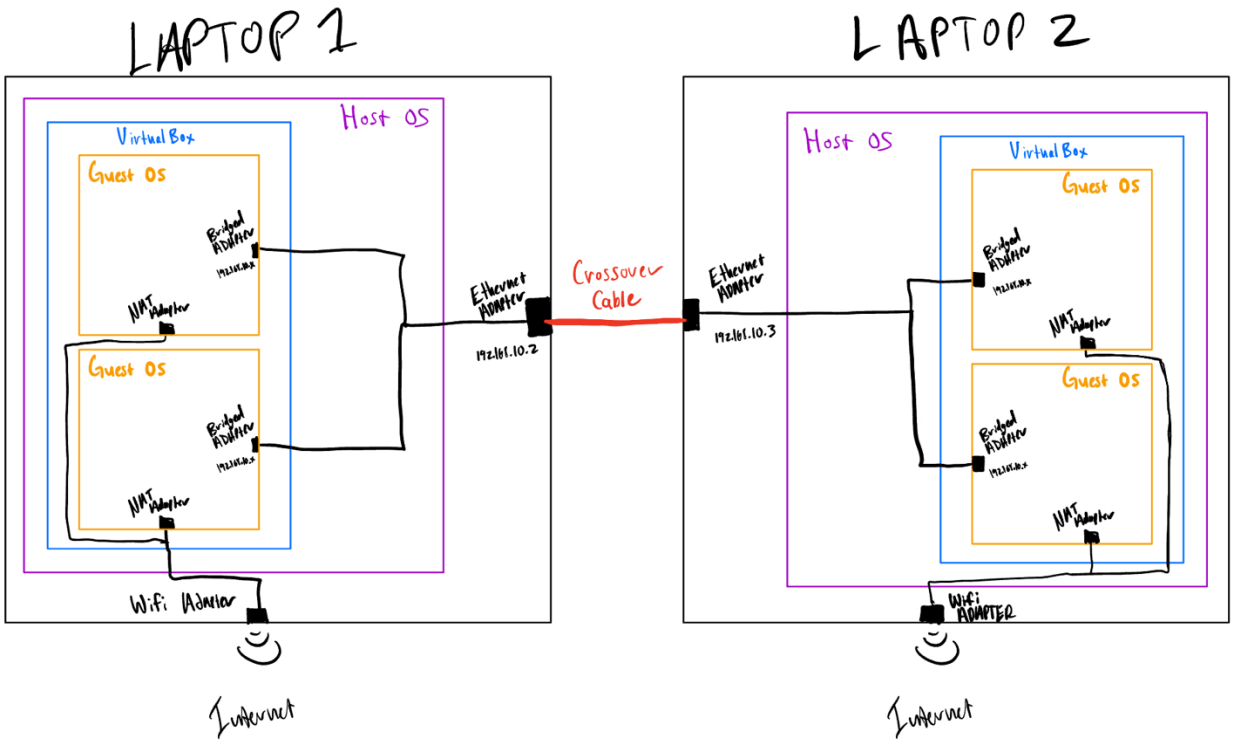
Initial Sketch



3/27/2026

Networking Scope

This Project will contain a total of four Virtual Machines (VMs). The Attack VM (Kali Linux) will be on the same subnet as the rest of the LAN, simulating a internal attack. Both host machines will have two VMs with Host 1 having Windows (Agent) and Linux (Attacker), and Host 2 will have Ubuntu (SIEM) and Linux (Agent). The two hosts will connect with an Ethernet cable. All VMs will have a bridged network adapter to allow communication between them; they will also have a NAT adapter to reach the internet (mostly for downloading/updating applications)



Updated Networking Sketch ^

3/27/2026

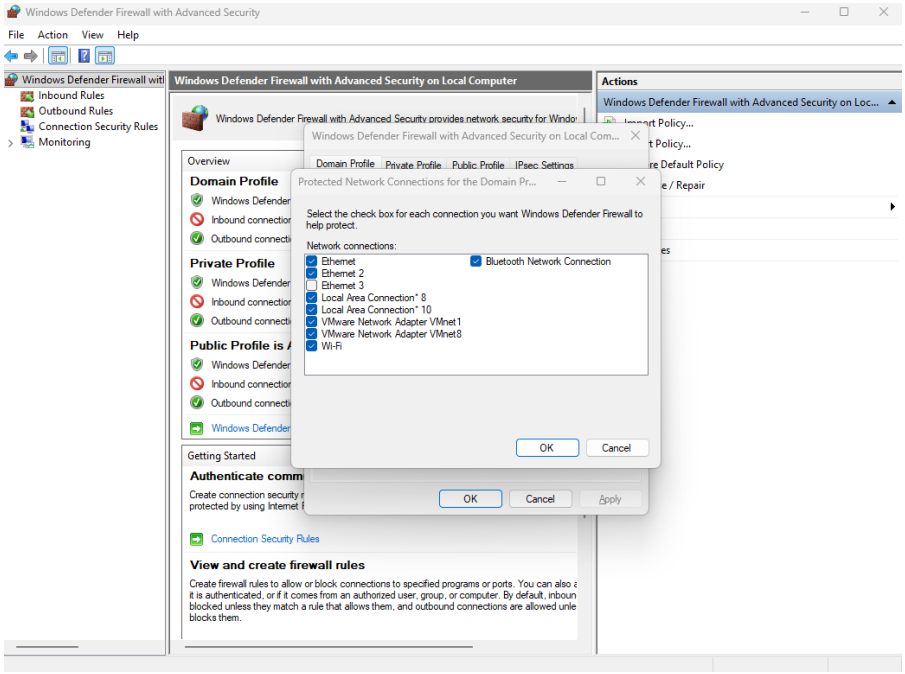
For an NIDS (Network Intrusion Detection System), we will be running Suricata on the Kali VM (since it doesn't work on windows), since we want the experience to see how a NIDS works, and because its very seamless with Wazuh.

For the IDS/IPS, we will be configuring SNORT on all of the devices because it works on everything, however we wont be able to aggregate the logs to Wazuh, so we will be using SNORTS main interface on the SIEM VM.

Machine Configuration

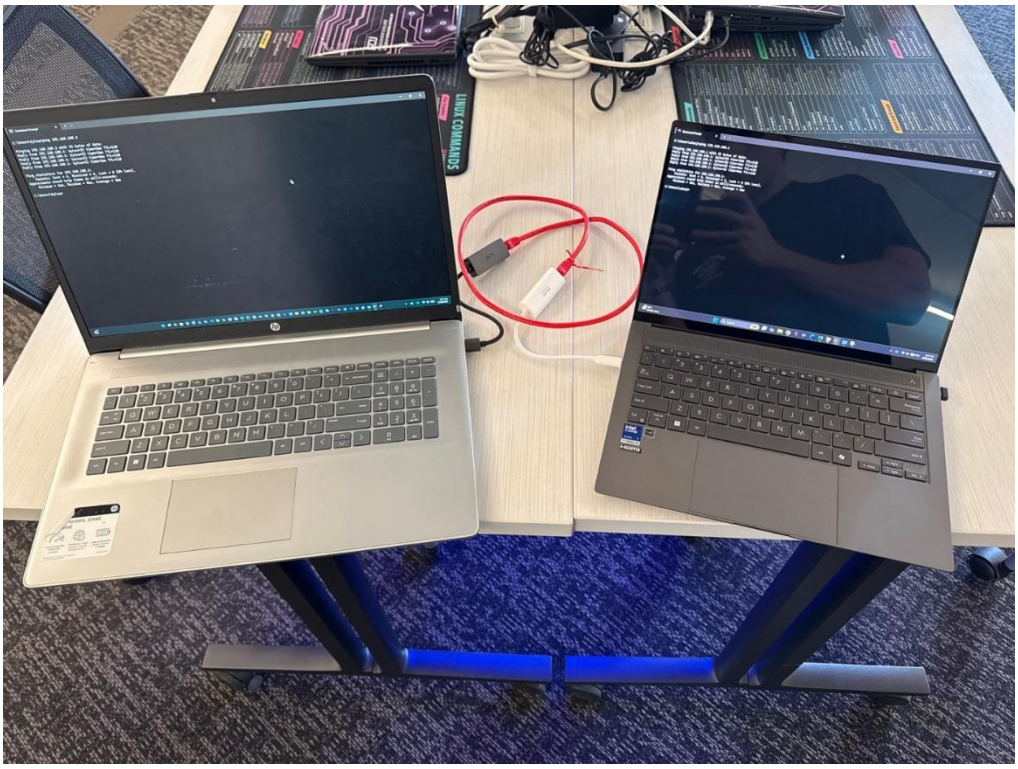
Host Machines

The first step is making our host machines communicate. When pinging the two machines (192.168.100.x /24) We were having trouble sending/receiving the ICMP request. We resolved this by disabling the ethernet windows defender setting on both host machine's ethernet adapters.



Firewall disabled for ethernet adapter

This isn't a security concern since the ethernet adapters we disabled in the firewall settings are only for the LAN connection between the two computers, and the outbound connection to the internet (and all other network adapters) is still protected by the windows firewall.



PING SUCCESSFUL! ^

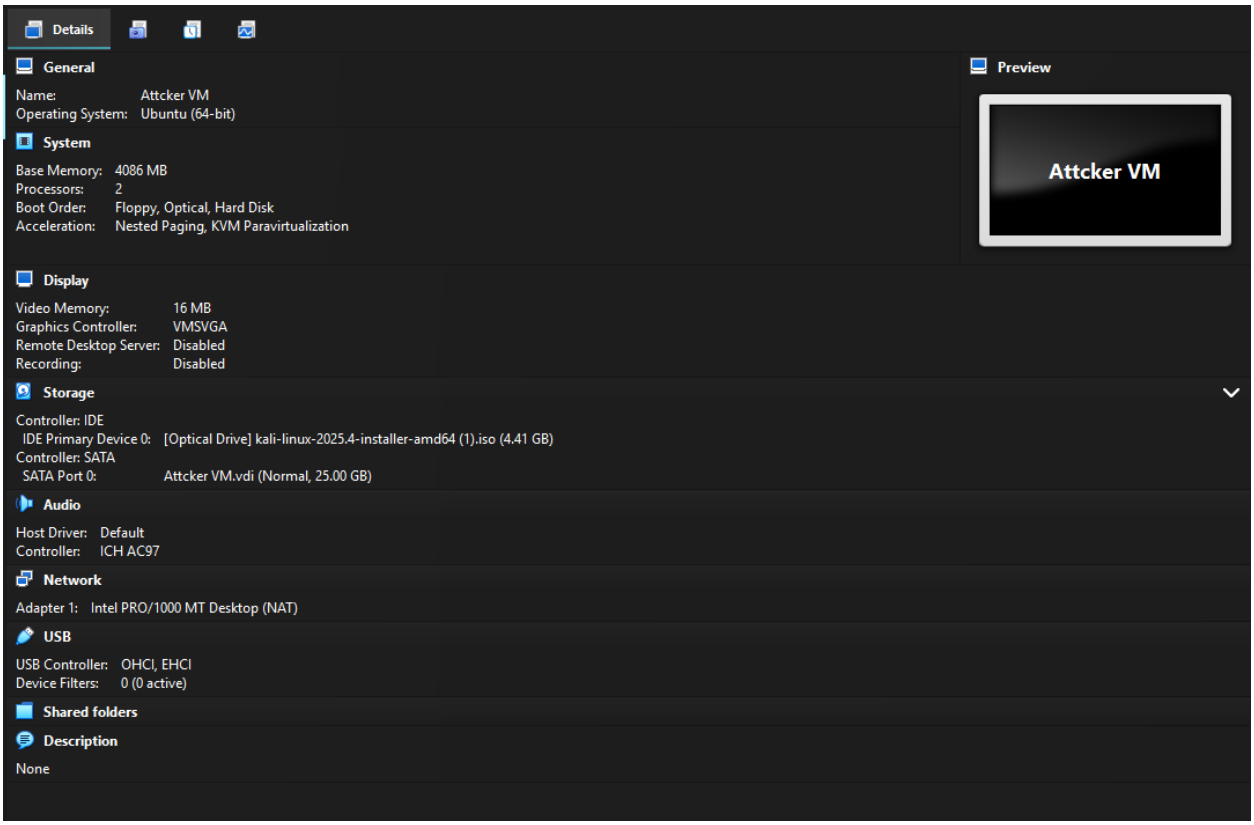
After we were able to ping each other's host machines, we then went on to make sure that all VM's can ping each other, as well as the hosts (6 devices total on the private LAN we created!)

```
C:\Users\adamj>arp -a

Interface: 192.168.100.2 --- 0xd
  Internet Address      Physical Address      Type
  192.168.100.1         00-e0-4c-68-4b-db     dynamic
  192.168.100.10        08-00-27-83-bd-57     dynamic
  192.168.100.11        08-00-27-31-8f-79     dynamic
  192.168.100.20        08-00-27-3f-20-4d     dynamic
```

ARP table entries of all VM's and Hosts! ^

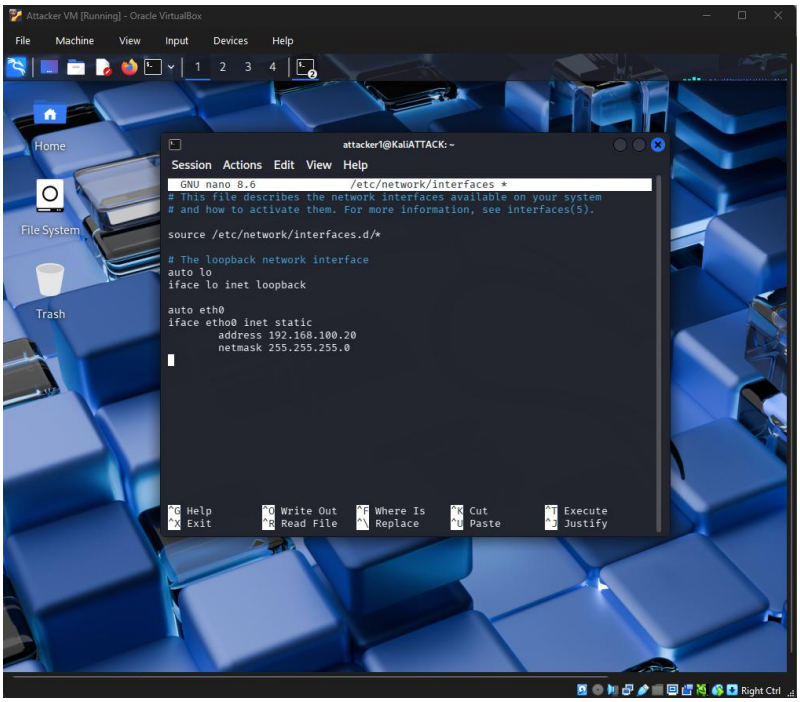
Attacker VM

- VirtualBox Kali-Linux VM Specifications



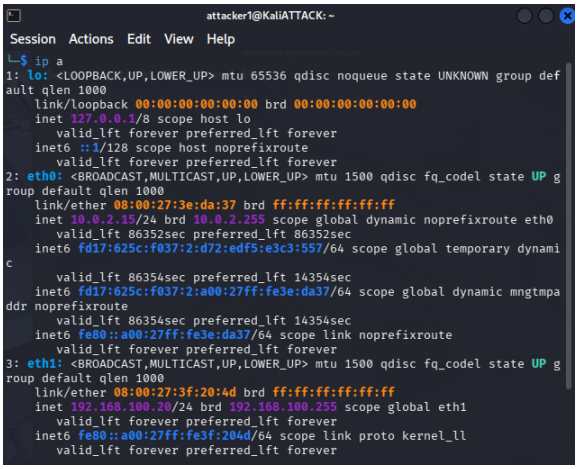
- Computer Information
    - Host name: KaliATTACK
    - Full name: DeAndre King
    - Username: attacker1
    - Password: kaliKali49
    - IP Address: 192.168.100.20

Assigning the Attakcer VM a static ip on the same subnet is what we decided to do. We assigned it the following IP 192.168.100.20 by opening the /etc/network/interface folder as sudo and adding the address and netmask.

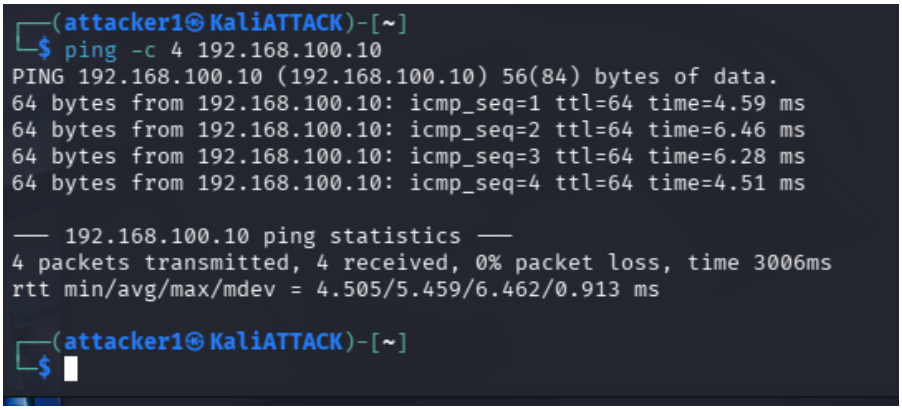


/etc/network/interfaces config file

We then restarted the network settings



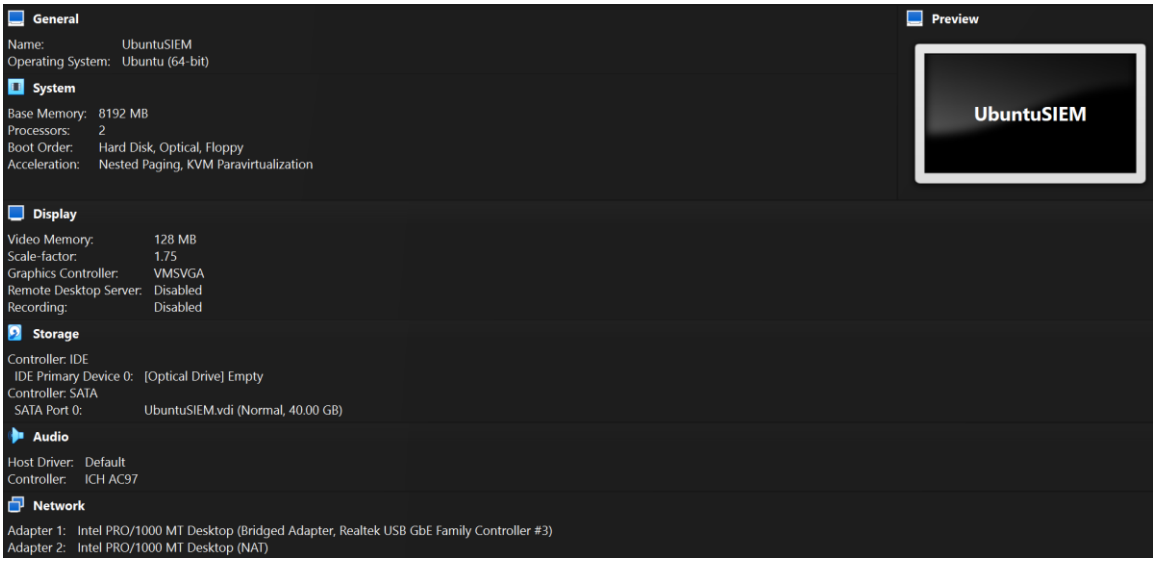
The ip has successfully been assigned the address ^



Kali VM (Attacker) on host 1 successfully pinged kali VM(SIEM) on host 2 ^

SIEM VM

- VirtualBox Ubuntu Desktop VM Specifications



- Computer Information
    - Host name: UbuSIEM
    - Full Name: Samuel Therington
    - Username: ubusiem
    - Password: kaliKali49
    - IP Address: 192.168.100.11

First thing we had to do was ensure that both the NAT adapter and the Bridged Network adapter was properly set up. This setup is very simple, since everything can be done through a GUI. Down below is the bridged adapter setup, making sure its on the right subnet so it can ping all devices.

Adapter 2 (Bridged Adapter) Settings

```
enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.100.11  netmask 255.255.255.0  broadcast 192.168.100.255
```

Ifconfig confirmation ^

After we were able to get VM to VM communication on one host, we then made sure that we can communicate from VM on host 1 to VM on host 2, as well as be able to ping the host from the VM, vise versa. This makes it so that we have a fully functional LAN with a total of 6 devices!

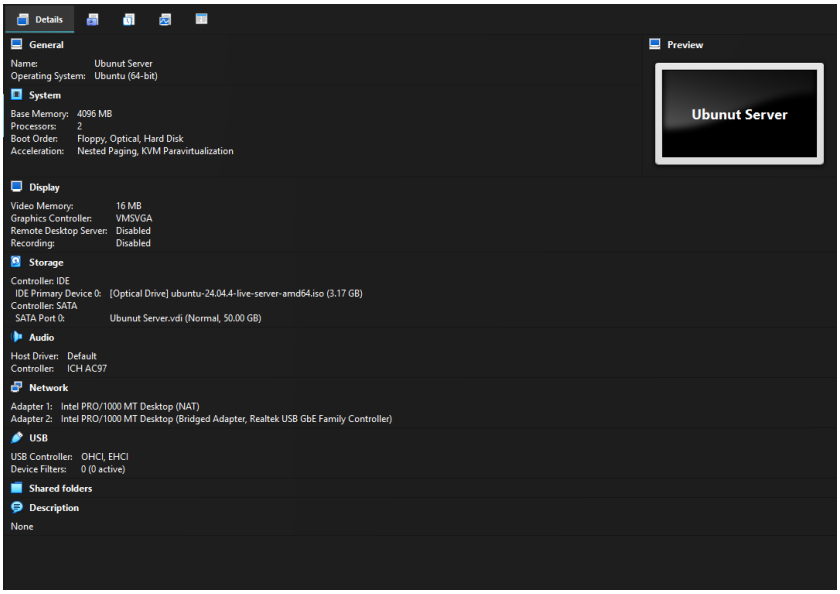
```
ubuserver@UbuntuSIEM:~$ arp -a
? (192.168.100.12) at 08:00:27:14:78:3b [ether] on enp0s8
_gateway (10.0.2.2) at 52:55:0a:00:02:02 [ether] on enp0s3
? (192.168.100.10) at 08:00:27:83:bd:57 [ether] on enp0s8
? (192.168.100.1) at 00:e0:4c:68:4b:db [ether] on enp0s8
? (192.168.100.2) at 4c:ea:41:68:af:32 [ether] on enp0s8
```

ARP table entries of all devices on the LAN ^

Snort Server

Ubuntu Server

- VirtualBox Ubuntu-Server VM Specifications



- Computer Information
    - Host name: snortubuntu1
    - Full name: Aubrey Graham
    - Username: snortserver
    - Password: sniffer50
    - IP Address: 192.168.100.30

We configured the ubuntu server and assigned it a static IP address from our LAN. We configured 2 NICs On for the management and one for Snort Sniffing. both using bridged adapter. We enable OpenSSH on this server to allow our other machines to remote into it.

```
snortserver@snortubuntu1:~$ sudo ip a
[sudo] password for snortserver:
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:96:4d:c9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.30/24 brd 192.168.100.255 scope global enp0s3
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe96:4dc9/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 08:00:27:c7:ba:ff brd ff:ff:ff:ff:ff:ff
```

Successful NIC configuration and assigned LAN IP address! ^

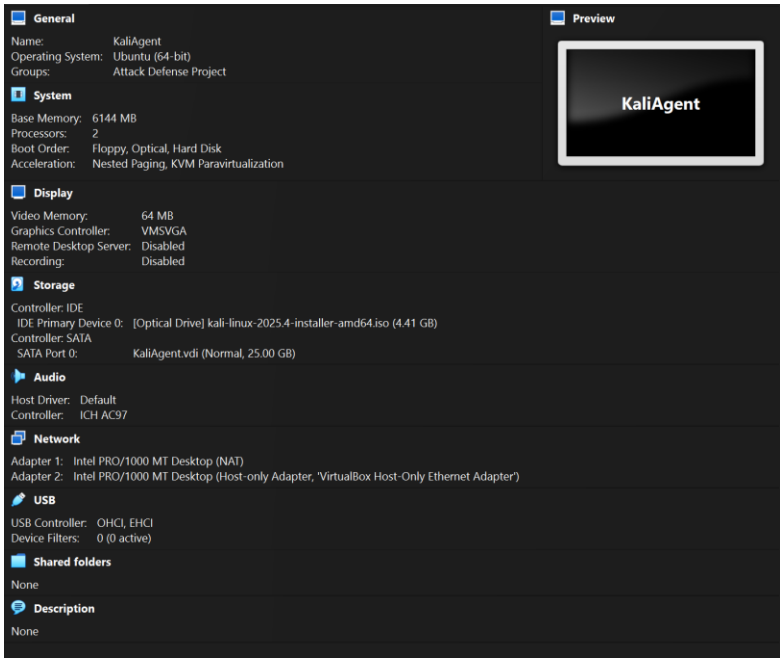
```
snortserver@snortubuntu1:~$ ping 192.168.100.11
PING 192.168.100.11 (192.168.100.11) 56(84) bytes of data.
64 bytes from 192.168.100.11: icmp_seq=1 ttl=64 time=10.0 ms
64 bytes from 192.168.100.11: icmp_seq=2 ttl=64 time=6.08 ms
64 bytes from 192.168.100.11: icmp_seq=3 ttl=64 time=5.95 ms
64 bytes from 192.168.100.11: icmp_seq=4 ttl=64 time=9.51 ms
64 bytes from 192.168.100.11: icmp_seq=5 ttl=64 time=7.14 ms
64 bytes from 192.168.100.11: icmp_seq=6 ttl=64 time=5.55 ms
64 bytes from 192.168.100.11: icmp_seq=7 ttl=64 time=5.77 ms
64 bytes from 192.168.100.11: icmp_seq=8 ttl=64 time=5.27 ms
64 bytes from 192.168.100.11: icmp_seq=9 ttl=64 time=7.63 ms
64 bytes from 192.168.100.11: icmp_seq=10 ttl=64 time=18.5 ms
```

Successfully pinged Host 2 VM ^

Agent VM's

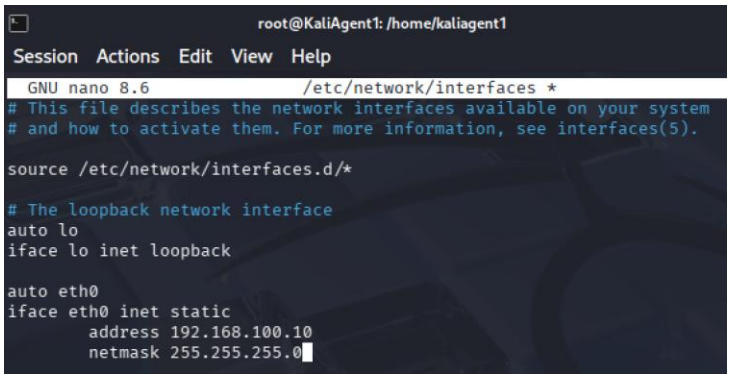
Kali Agent

- VirtualBox Kali-Linux VM Specifications

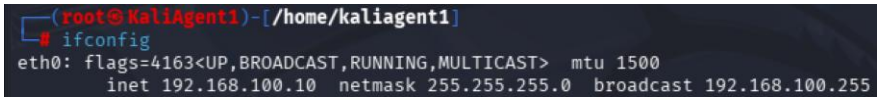


- Computer Information
  - Host name: KaliAgent1
  - Full name: Daniel Smith
  - Username: kaliagent1
  - Password: kaliKali49
  - IP Address: 192.168.100.10

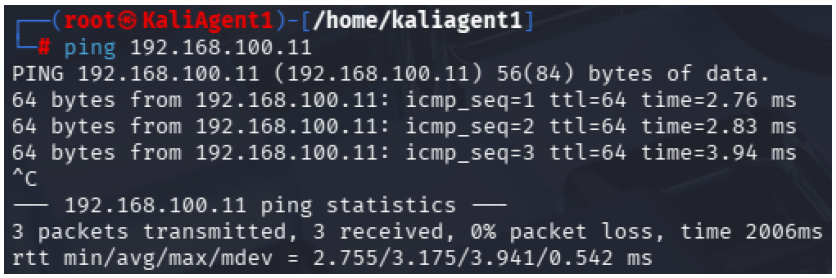
First thing we need to do is set a static IP address for the VM. We do this by editing the /etc/network/interfaces config file. Below is the proper configuration for this file! After, we made sure that the ping worked from VM to VM on a single host, before we did cross host VM to VM communication!



/etc/network/interfaces config file ^

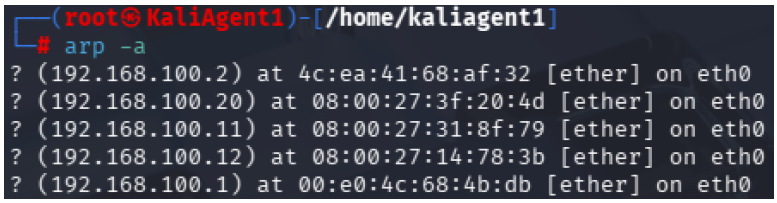


Ifconfig confirmation ^



Internal Host VM <-> VM Ping Confirmation ^

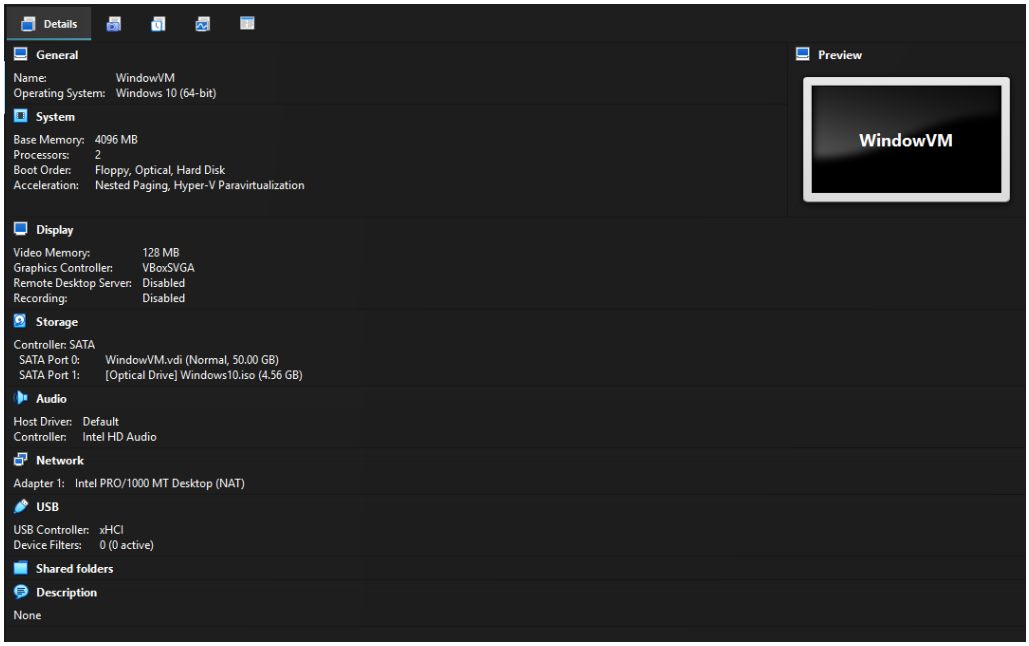
After we were able to get VM to VM communication on one host, we then made sure that we can communicate from VM on host 1 to VM on host 2, as well as be able to ping the host from the VM, vise versa. This makes it so that we have a fully functional LAN with a total of 6 devices!



ARP table entries of all devices on the LAN ^

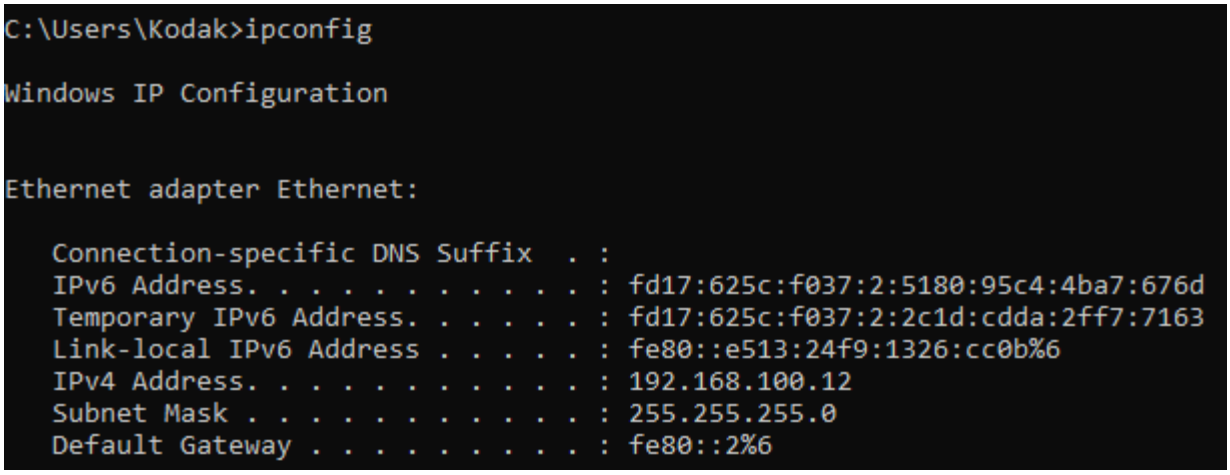
Windows Agent

- VirtualBox Window's 10 VM Specifications



- Computer Information
  - Host name: WindowsAgent
  - Full name: Kodak Black
  - Username: windowsagent1 (KodakBlack288@outlook.com)
  - Password: windows49 (5544)
  - IP Address: 192.168.100.12

When launching the windows VM, I ran an IP config command to see what the default IP address for VM is. We will be assigning a static IP that's located in the same subnet.



Static IP Set above ^

After assigning the Windows agent with a IP in the LAN subnet we ran a ping command to all the other 5 devices inside of the LAN. After running the Ping we performed an ‘arp -a’ command showing all the IP addresses and MAC addresses inside of the LAN.

```
C:\Users\Kodak>ping 192.168.100.11

Pinging 192.168.100.11 with 32 bytes of data:
Reply from 192.168.100.11: bytes=32 time=14ms TTL=64
Reply from 192.168.100.11: bytes=32 time=6ms TTL=64
Reply from 192.168.100.11: bytes=32 time=7ms TTL=64
Reply from 192.168.100.11: bytes=32 time=5ms TTL=64

Ping statistics for 192.168.100.11:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 5ms, Maximum = 14ms, Average = 8ms
```

Host 1 Windows VM successfully pinged host 2 Kali (SIEM) VM ^

```
C:\Users\Kodak>arp -a

Interface: 10.0.3.15 --- 0x5
  Internet Address      Physical Address      Type
  10.0.3.2              52-55-0a-00-03-02     dynamic
  10.0.3.255            ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

Interface: 192.168.100.12 --- 0x6
  Internet Address      Physical Address      Type
  192.168.100.1         00-e0-4c-68-4b-db     dynamic
  192.168.100.10        08-00-27-83-bd-57     dynamic
  192.168.100.11        08-00-27-31-8f-79     dynamic
  192.168.100.20        08-00-27-3f-20-4d     dynamic
  192.168.100.255       ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
```

‘ARP -a’ table entries of all devices on the LAN ^

Blue Team

Defense Summary (do at end)

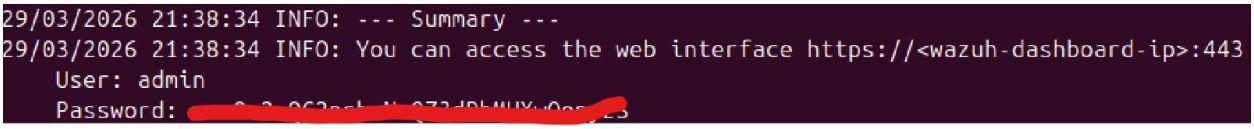
In this project, the blue team designed and implemented a multi-layered defensive environment across multiple virtual machines to simulate a realistic enterprise network. We deployed Wazuh as our centralized SIEM to aggregate, correlate, and analyze security events, providing real-time visibility into system and network activity. To strengthen network-based threat detection, we integrated both Snort and Suricata as IDS solutions, leveraging their signature-based capabilities to identify malicious traffic. Additionally, we developed a comprehensive custom Snort ruleset tailored to our lab environment, which streamlined alert categorization and improved log readability when ingested into Wazuh. This integration allowed us to efficiently monitor, detect, and respond to simulated attacks, demonstrating effective coordination between host-based and network-based defense mechanisms.

Step-By-Step

Wazuh

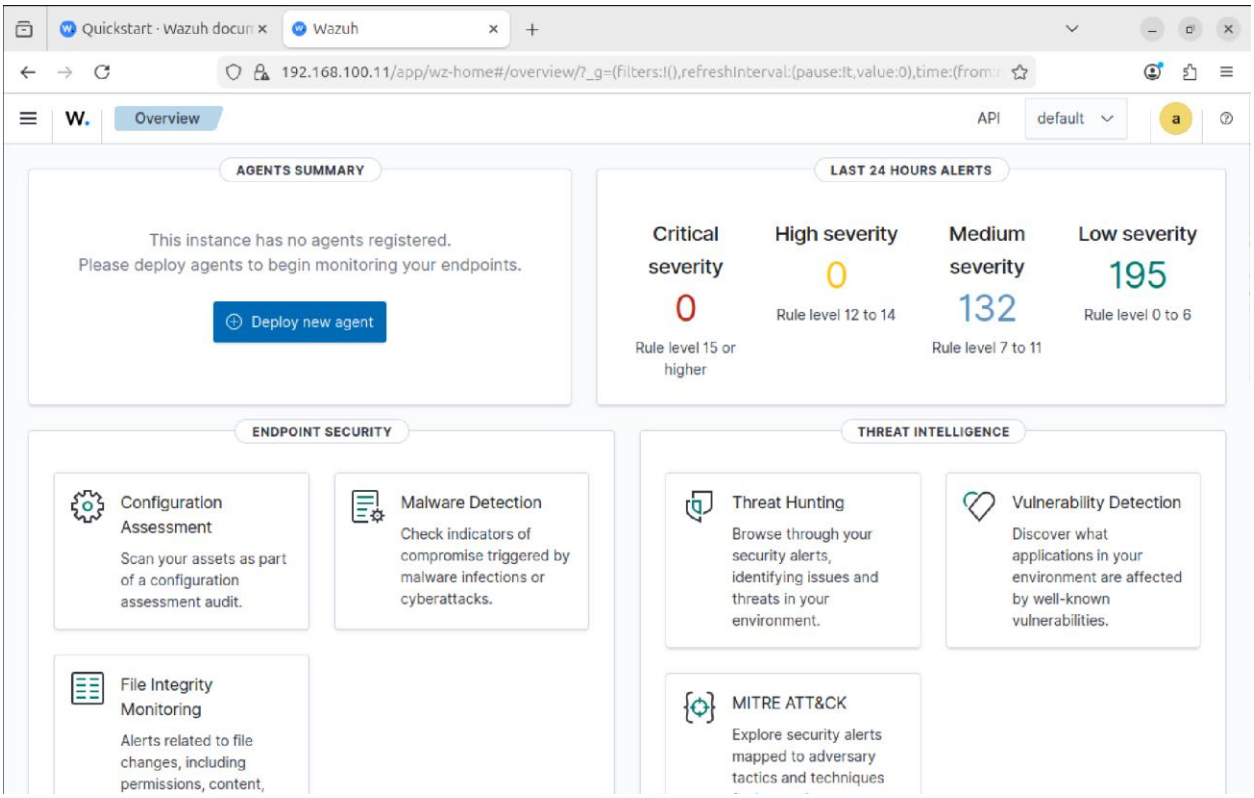
First thing we will do is install the SIEM dashboard on the SIEM VM, and then the agents on both the Kali and Windows machine. For this, we will need internet access, so its important to make sure that your VM's have a NAT adapter enabled.

First, go to the Wazuh website and click on the installation guide. Goto the quickstart installation guide, and copy and paste the quickstart command (curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a) This will install all 3 parts of wazuh: the indexer, manager, and dashboard. After the installation is done, there will be an admin username and password, make sure to mark those down for logging into your admin dashboard.



Wazuh Login Credentials ^

Next, copy the web interface link and make sure you edit your IP address to your host machines IP address, and paste it into your preferred browser. Now that we have the dashboard up and running, we will now begin to install the agents on the other virtual machines.

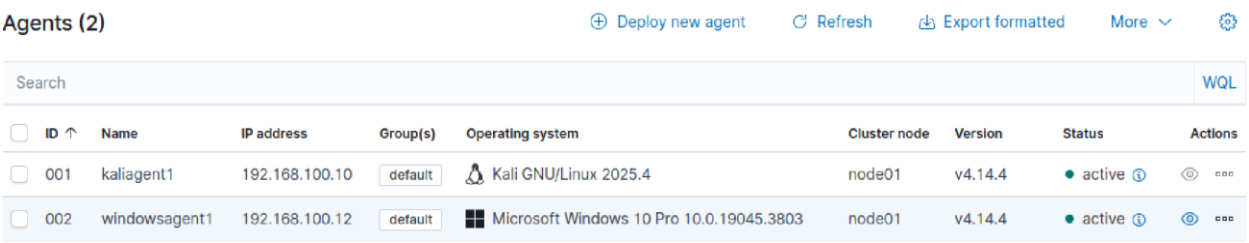


Wazuh Dashboard ^

To install agents on the other virtual machines, go to the deploy new agents on the Wazuh dashboard, and fill out the agent information. After, you will get a command to copy and paste into the terminal, and it will automatically install and sync to your server! Just make sure to run the systemctl enable and start commands for the agent to be activated!

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.4-1_amd64.deb && sudo WAZUH_MANAGER='192.168.100.11'
WAZUH_AGENT_NAME='kaliagent1' dpkg -i ./wazuh-agent_4.14.4-1_amd64.deb
```

Agent command example^



Active Agents ^

Now that we have Snort3 Integrated and working properly with Wazuh, we created an entire rules file that have all of our custom rules that will make the attack section a lot more easy to read, as well as easy to follow because all of our IP's are very similar, and we want the logs to be a lot more visually appealing! Down below is where you can find a small portion of our newly

created rules .xml file. As you can see, they have specific descriptions based off what the attack is, as well as having level of severity based on how severe the attack is!

```
<group name="snort,">

 <rule id="100200" level="6">
   <decoded_as>snort3</decoded_as>
   <description>Snort3 IDS alert detected</description>
 </rule>

 <rule id="100201" level="10">
   <if_sid>100200</if_sid>
   <match>192.168.100.20</match>
   <description>Snort3: Traffic detected from attacker VM</description>
 </rule>

 <rule id="100202" level="12">
   <if_sid>100201</if_sid>
   <match>port_scan</match>
   <description>Snort3: Port scan from attacker VM 192.168.100.20</description>
 </rule>

 <rule id="100203" level="12">
   <if_sid>100201</if_sid>
   <match>ssh</match>
   <description>Snort3: SSH brute force from attacker VM 192.168.100.20</description>
 </rule>

 <rule id="100204" level="14">
   <if_sid>100201</if_sid>
   <match>dos</match>
   <description>Snort3: SYN flood DoS from attacker VM 192.168.100.20</description>
 </rule>

 <rule id="100205" level="10">
   <if_sid>100201</if_sid>
   <match>arp_spoof</match>
   <description>Snort3: ARP spoofing from attacker VM 192.168.100.20</description>
```

Snort3_rules.xml file snippet ^

	Apr 18, 2026 @ 20:43:10.720	ubuntusnort	Snort3: Port scan from attacker VM 192.168.100.20	12	100202

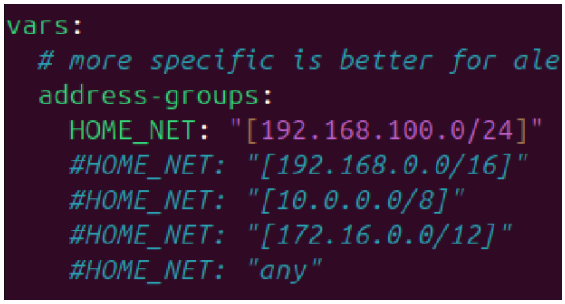
Successful Port Scan Alert ^

	↓ timestamp	agent.name	rule.description	rule.level	rule.id
	Apr 21, 2026 @ 01:11:16.859	UbuntuSIEM	Suricata: Alert - SURICATA ICMPv4 unknown code	3	86601
	Apr 21, 2026 @ 01:11:16.859	UbuntuSIEM	Suricata: Alert - SURICATA ICMPv4 unknown code	3	86601
	Apr 21, 2026 @ 01:11:16.401	ubuntuagent1	Suricata: Alert - SURICATA ICMPv4 unknown code	3	86601
	Apr 21, 2026 @ 01:11:16.401	ubuntuagent1	Suricata: Alert - SURICATA ICMPv4 unknown code	3	86601
	Apr 21, 2026 @ 01:11:15.981	ubuntuagent1	Suricata: Alert - SURICATA ICMPv4 unknown code	3	86601
	Apr 21, 2026 @ 01:11:15.930	ubuntuagent1	Suricata: Alert - SURICATA ICMPv4 unknown code	3	86601
	Apr 21, 2026 @ 01:11:15.357	UbuntuSIEM	Suricata: Alert - SURICATA ICMPv4 unknown code	3	86601
	Apr 21, 2026 @ 01:11:15.354	UbuntuSIEM	Suricata: Alert - SURICATA ICMPv4 unknown code	3	86601
	Apr 21, 2026 @ 01:10:18.128	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 21, 2026 @ 01:10:18.067	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 21, 2026 @ 01:07:58.168	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 21, 2026 @ 01:07:58.100	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 21, 2026 @ 01:06:15.223	ubuntusnort	Snort3: Traffic detected from attacker VM	10	100201
	Apr 21, 2026 @ 01:06:15.218	ubuntusnort	Snort3: Traffic detected from attacker VM	10	100201
	Apr 21, 2026 @ 01:06:15.214	ubuntusnort	Snort3: Traffic detected from attacker VM	10	100201

Suricata and Snort Alerts ^

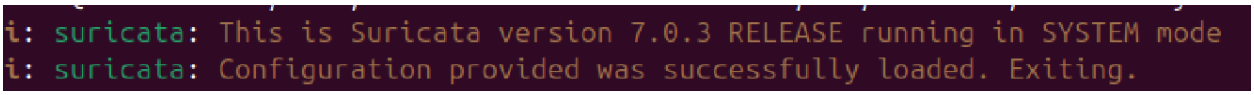
Suricata (NIDS)

Next, we will be installing and configuring a NIDS on the Kali Agent. Sadly, Suricata doesn't work on Windows, so we can only use it on the Kali VM. The first thing we are going to do is to install Suricata on the SIEM VM. Very basic installation, just a sudo apt install Suricata! After, you need to go into the /etc/suricata/suricata.yaml file and edit the HOME_NET to your LAN subnet! Also, make sure in the af packet: section, the interface is the same as your bridged network adapter. In my case, I had to change it from eth0 to enp0s8.



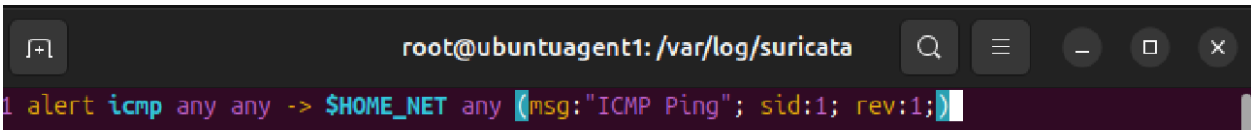
.yaml file config^

After, you are going to run a test command to make sure that there is no errors created from editing the .yaml file. Run a suricata -T -c /etc/suricata/suricata.yaml and make sure you get a successful message after!



Successful Test^

I wanted to create a custom rule for fun, so I made a basic ICMP rule with an sid of 1 and to match it with any source address so that any computer that pings the agents IP will get logged. Down below is the syntax you want to put into the /var/log/suricata/local.rules file. After, you want to make sure you update the suricata rules so it knows to use it by going to the .yaml file and going to the rules-files list, and adding the path to your new rules file!



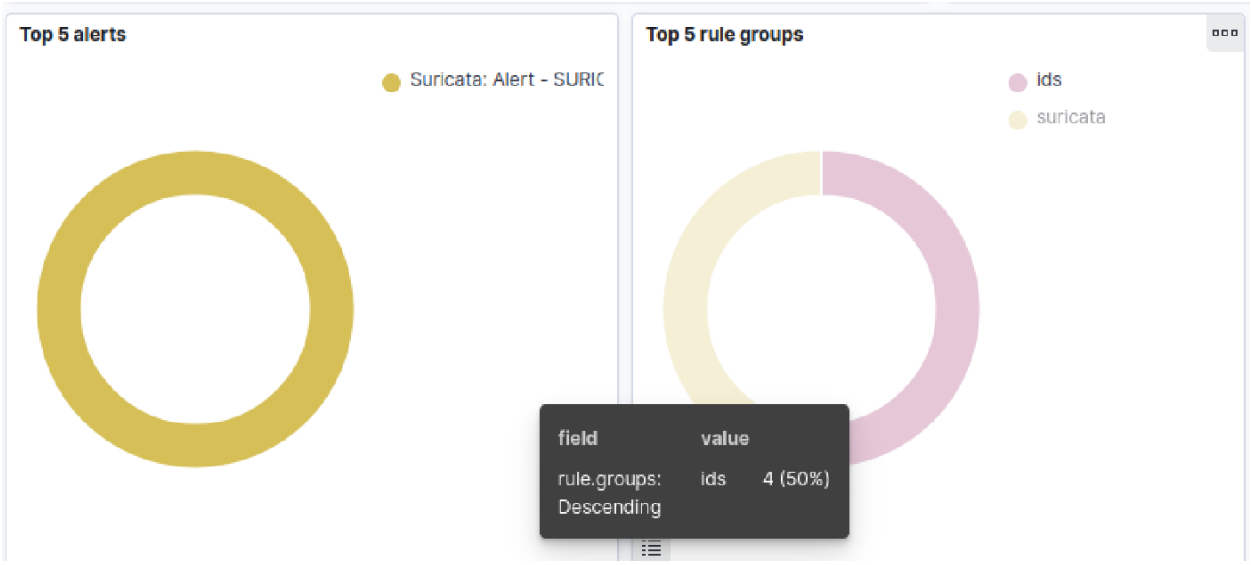
Custom ICMP Ping Rule ^

To make sure that the logs are being sent to the wazuh SIEM in correct format, you want to make sure that you goto the wazuh manager config (ossec.conf file) and make sure that you have the log format to json, and the location to /var/log/suricata/eve.json to ensure that it is able to take the suricata logs and use the json decoder built into wazuh.

```
335   <localfile>
336     <log_format>json</log_format>
337     <location>/var/log/suricata/eve.json</location>
338   </localfile>
339
```

Successful Ossec.conf ^

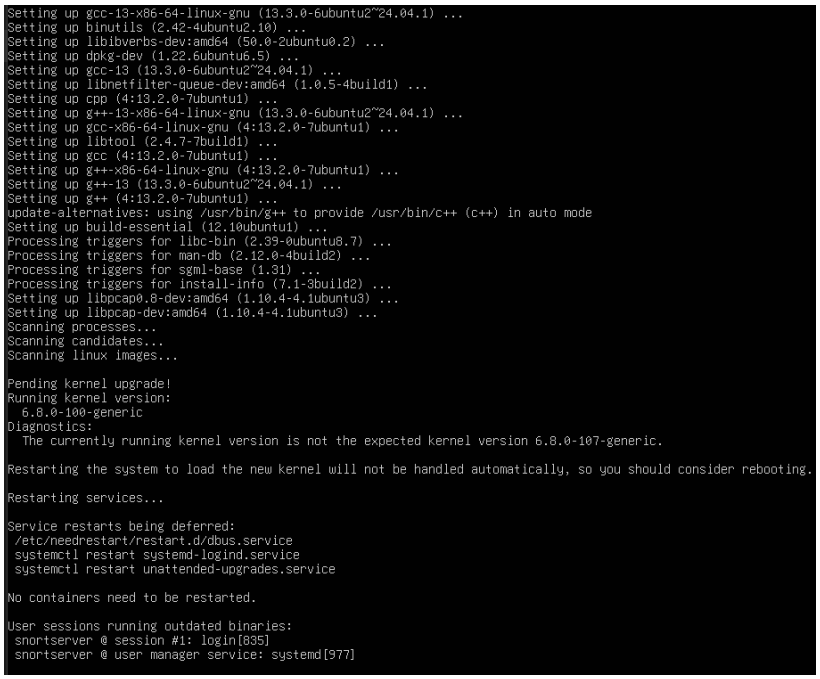
Now that we have everything properly configured, make sure to restart both suricata, and wazuh agent/dashboard. Now for a quick test, send a ping request to your agent and then check the wazuh events tab, you should see the suricata alert pop up!



Successful Suricata Log ^

SNORT (IDS/IPS)

After successfully pinging the VMs and hosts inside the LAN we were able to also ping 8.8.8.8 confirming we had internet access on the machine. We then ran updates on the server and began installation of Snort 3, this is going to be our network IDS/IPS.We began with installing all dependencies. We then rebooted and began LibDAQ installation.



Successful Installation of dependencies ^

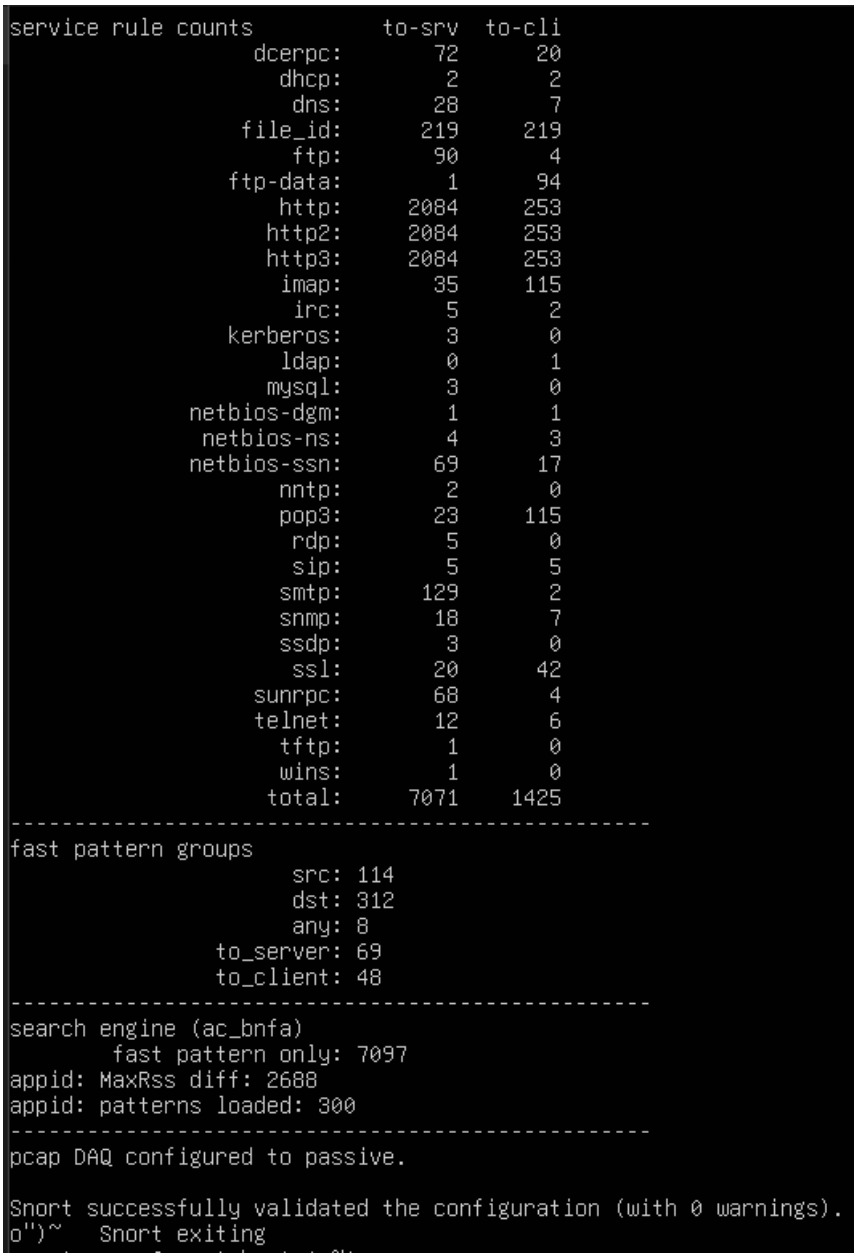
Successful installation of LibDAQ and Snort 3! ^

```
-- HOME_NET and EXTERNAL_NET must be set now
-- setup the network addresses you are protecting
HOME_NET = '192.168.100.0/24'

-- set up the external network addresses.
-- (leave as "any" in most situations)
EXTERNAL_NET = 'any'

include 'snort_defaults.lua'
```

Configured the Snort rules to detect any traffic inside our LAN ^

```
ips =
{
    enable_builtin_rules = true,
    include = '/usr/local/etc/rules/snort3-community-rules/snort3-community.rules',
    variables = default_variables
}
```

Activating Snort engine to allow traffic analysis and generate logs ^



Validation of 8496 pre-configured rules ^

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.4-1_amd64.deb && sudo WAZUH_MANAGER='192.168.100.11'
WAZUH_AGENT_NAME='ubuntusnort' dpkg -i ./wazuh-agent_4.14.4-1_amd64.deb
```

ⓘ Requirements

- You will need administrator privileges to perform this installation.
- Shell Bash is required.

Keep in mind you need to run this command in a Shell Bash terminal.

Start the agent:

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

Copy command

Command to install the Wazuh Agent on the Snort Server ^

After successful installation of the agent on the server, we then told the agent VM to read all logs inside of the alert_fast.txt file and display it inside of the wazuh dashboard.

```
  <localfile>
    <log_format>snort-fast</log_format>
    <location>/var/log/snort/alert_fast.txt</location>
  </localfile>

</ossec_config>
```

Cmd telling the agent to read log files inside of the alert_fast.txt file ^

Now we have the file path that has all the detection rules inside of it, if we need to change any detection rules or create them, we open the /etc/systemd/system/snort3.services file to do so. We also added a rule for ubuntu to start on snort3 whenever the machines boot up.

```
snortserver@snortubuntu1:~$ sudo systemctl cat snort3
# /etc/systemd/system/snort3.service
[Unit]
Description=Snort3 NIDS
After=network.target

[Service]
Type=forking
ExecStart=/usr/local/bin/snort -c /usr/local/etc/snort/snort.lua -i enp0s9 -A alert_fast -l /var/log/sno
Restart=on-failure

[Install]
WantedBy=multi-user.target
snortserver@snortubuntu1:~$ _
```

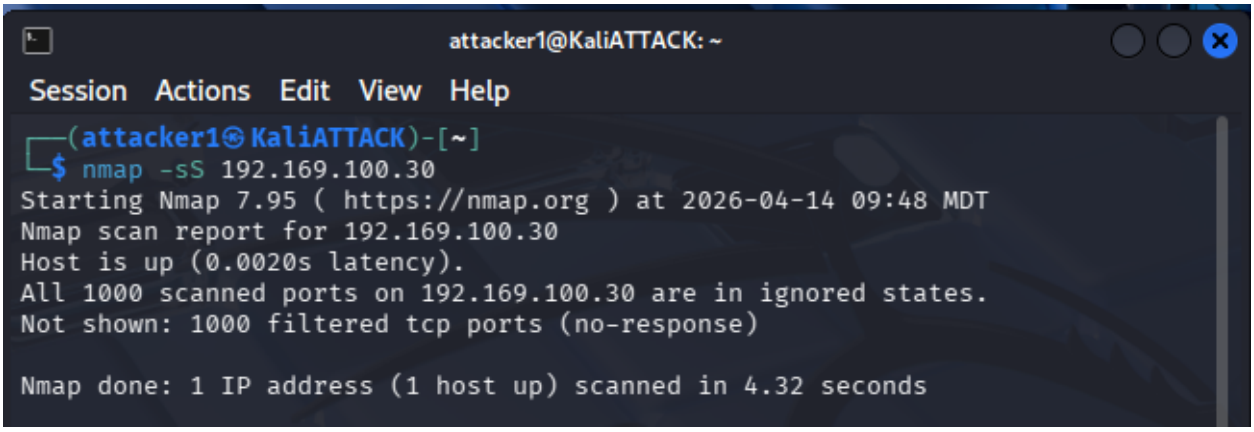
Command for snort3 to start on bootup on ubuntu ^

We then created a file /usr/local/etc/snort/local.rules that will have all custom rules for our environment. We created a custom rule to detect all activitie from the insider threat (attacker). We did this creating an alert rule for any port network activities from the attackers IP on any port to any IP and any Port

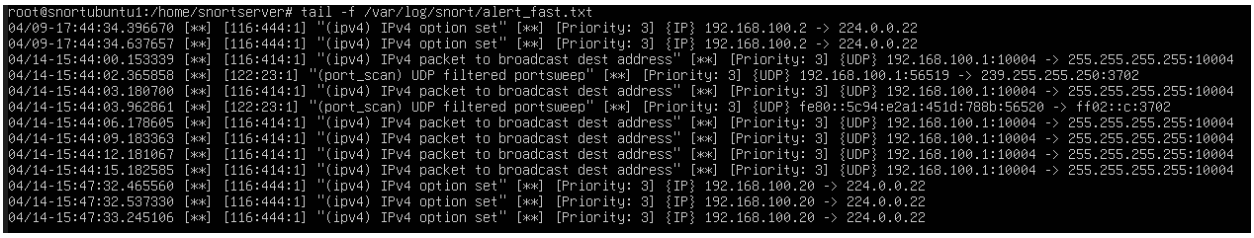
```
alert ip 192.168.100.20 any -> any any (msg:"ALERT: Activity from VM 192.168.100.20"; sid:1000001; rev:1;)
alert icmp 192.168.100.20 any -> any any (msg:'PING detected from VM 192.168.100.20"; sid:1000002; rev:1;)_
```

Custom rules for attacker IP ^

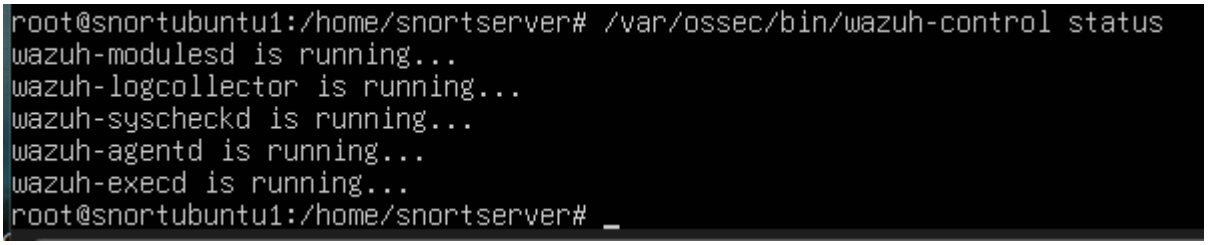
We tested to see if the rules would capture an nmap scan fromt the attacker VM. We ran an nmap –sS 192.169.100.30 scanning the snort VM for open ports



Nmap scan from the attacker VM to the snort VM ^



Nmap successfully getting captured inside of the alert_fast.txt file



Confirmed wazuh agent is live inside of the VM ^

Next, we had to make sure there was a snort3 .xml rules and decoder file created on the SIEM VM to make sure it was able to send the alerts to wazuh. After creating those two files, we then ran a basic NMAP scan on the ubuntu snort machine, and snort was able to take the logs that it got from the scan and send them over perfectly to Wazuh!

	↓ timestamp	agent.name	rule.description	rule.level	rule.id
	Apr 18, 2026 @ 20:09:03.753	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:09:03.75[illegible]	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:09:03.753	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:09:03.752	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:09:03.752	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:09:03.752	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:09:03.695	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:06:53.77[illegible]	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:06:53.732	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:06:53.726	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:06:53.726	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:06:53.721	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:06:53.721	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:06:53.720	ubuntusnort	Snort3 IDS alert detected	6	100200
	Apr 18, 2026 @ 20:06:53.720	ubuntusnort	Snort3 IDS alert detected	6	100200

Successful Port Scan Alert from Snort ^

Problems Occurred

- We tried to create a MISP Ubuntu VM that would make the Wazuh alerts a little more advanced by running it through IOC enrichment to get more correlation between logs, however when we got to the step of us having to integrate MISP and Wazuh, the API key

integration was out of our knowledge scope and we only had a couple more hours of working on the project before summer started and we split apart geographically!

- Fixed Wazuh agent connectivity and enrollment issues by checking agent and manager logs, verifying network reachability, and confirming the Wazuh services were actually running when agents were disconnected or failing to enroll.
- Troubleshot Wazuh dashboard/indexer problems by checking dashboard logs, confirming the dashboard and indexer services were active, and verifying the vulnerability/indexer configuration and indexes were created correctly.
- Resolved Suricata setup and alerting issues by validating the install, making sure the rule set loaded correctly, and checking Suricata log output like eve.json when alerts or detections weren't showing up as expected.
- Debugged Suricata rule conflicts or misconfigurations by verifying signatures were unique and that detection rules were formatted correctly so Suricata would start and process traffic normally.
- Snort 3 Configuration Issue: Snort was compiled from source rather than installed via apt, meaning default config paths didn't exist and the service file required multiple fixes including changing Type=forking to Type=simple and correcting the network interface from enp0s9 to enp0s3 before it would run reliably as a systemd service.
- Wazuh Agent Connectivity: The Wazuh agent on the Snort VM repeatedly failed to connect to the manager at 192.168.100.11:1514 whenever the Wazuh Manager VM was powered off, requiring manual restarts and highlighting the importance of keeping all VMs running simultaneously during testing.
- Custom Decoder and Rules Placement: Snort alerts were not appearing on the Wazuh dashboard because the custom decoder and rules files were created on the agent VM instead of the Wazuh Manager VM, where all log parsing and alert generation actually takes place.
- Regex Syntax Errors in Wazuh Decoder: The initial prematch regex used in the Snort decoder caused repeated syntax errors and prevented the Wazuh Manager from starting, requiring simplification of the regex pattern to match Snort 3's specific alert log format before alerts successfully flowed through to the dashboard.

Red Team

Attack Summary (do at end)

This project simulated a series of real-world cyberattacks against a victim Kali Linux machine at 192.168.100.10 within a private SOC lab environment, with all attacks successfully detected by **Snort 3 IDS** and forwarded to the **Wazuh SIEM** dashboard for analysis.

The attack phase began with **network reconnaissance** using Nmap to perform a SYN port scan and OS fingerprinting, revealing three open services — FTP (21), SSH (22), and HTTP (80) — and identifying the target as a Linux system, giving the attacker a clear picture of the attack surface.

A **brute force attack** was then launched against both the SSH and FTP services using Hydra with the rockyou wordlist, attempting millions of credential combinations to gain unauthorized access to the victim machine.

A **SYN flood Denial of Service attack** was executed using hping3, transmitting over 343,000 packets to the victim's web server on port 80, resulting in 100% packet loss and simulating a real-world service disruption scenario.

An **ARP spoofing attack** was carried out using arpspoof, poisoning the victim's ARP cache and positioning the attacker as a Man-in-the-Middle between the victim and the network gateway, allowing interception of network traffic.

Finally a **Nikto web vulnerability scan** identified multiple misconfigurations on the victim's Apache server including missing security headers, exposed HTTP methods, and a known CVE mapped to information disclosure and XSS vulnerabilities.

All attacks were detected by **Snort 3 or Suricata** and correlated within the **Wazuh SIEM**, confirming the full detection pipeline was functioning as intended.

Step-By-Step

NMAP Recon

Attack Begins! This attack will simulate an inside threat performing recon then using that intel to exploit vulnerabilities. To begin, we wanted to run a Nmap -sS port scan for the initial recon to identify the open ports.

2

```
┌──(attacker1㉿KaliATTACK)-[~]
└─$ nmap -sn 192.168.100.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 19:06 MDT
Nmap scan report for 192.168.100.1
Host is up (0.00071s latency).
MAC Address: 00:E0:4C:68:4B:DB (Realtek Semiconductor)
Nmap scan report for 192.168.100.2
Host is up (0.0060s latency).
MAC Address: 4C:EA:41:68:AF:32 (Gopod Group Limited)
Nmap scan report for 192.168.100.10
Host is up (0.015s latency).
MAC Address: 08:00:27:83:BD:57 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Nmap scan report for 192.168.100.11
Host is up (0.012s latency).
MAC Address: 08:00:27:0A:0B:20 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Nmap scan report for 192.168.100.15
Host is up (0.013s latency).
MAC Address: 08:00:27:B2:D7:11 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Nmap scan report for 192.168.100.30
Host is up (0.0072s latency).
MAC Address: 08:00:27:96:4D:C9 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Nmap scan report for 192.168.100.20
Host is up.
Nmap done: 256 IP addresses (7 hosts up) scanned in 2.17 seconds
```

Successful NMAP scan of all hosts up ^

The insider threat decides to attack 192.168.100.10. The next step was to run a nmap –O to identify the fingerprint of the IP. This nmap scan was able to identify the OS of the victim along with 3 open ports: SSH on (22), HTTP on (80), and FTP on port 21.

```
┌──(attacker1㉿KaliATTACK)-[~]
└─$ nmap -O 192.168.100.10
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 19:25 MDT
Nmap scan report for 192.168.100.10
Host is up (0.0077s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
80/tcp open  http
MAC Address: 08:00:27:83:BD:57 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.32 seconds
```

Successful OS Fingerprint ^

Hydra Brute Force

Next, we decided to try a brute force attack on SSH, using Hydra and the RockYou.txt wordlist (classic). We obviously stopped it after while, but here is the command and also a look at what the SIEM detected from this attack. Suricata was the application (NIDS) that was able to recognize invalid SSH attempts, and its obvious that we are performing a brute force attack due to the large volume of invalid attempts.

```
┌──(root㉿KaliATTACK)-[/usr/share/wordlists]
└─# hydra -l root -P rockyou.txt 192.168.100.10 ssh -t 4 -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizati

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-20 19:33:51
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.100.10:22/
[ATTEMPT] target 192.168.100.10 - login "root" - pass "123456" - 1 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "12345" - 2 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "123456789" - 3 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "password" - 4 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "iloveyou" - 5 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "princess" - 6 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "1234567" - 7 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "rockyou" - 8 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "12345678" - 9 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "abc123" - 10 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "nicole" - 11 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "daniel" - 12 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "babygirl" - 13 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "monkey" - 14 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "lovely" - 15 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "jessica" - 16 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "654321" - 17 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "michael" - 18 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "ashley" - 19 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "qwerty" - 20 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "111111" - 21 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "iloveu" - 22 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "000000" - 23 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "michelle" - 24 of 14344399 [child 0] (0/0)
```

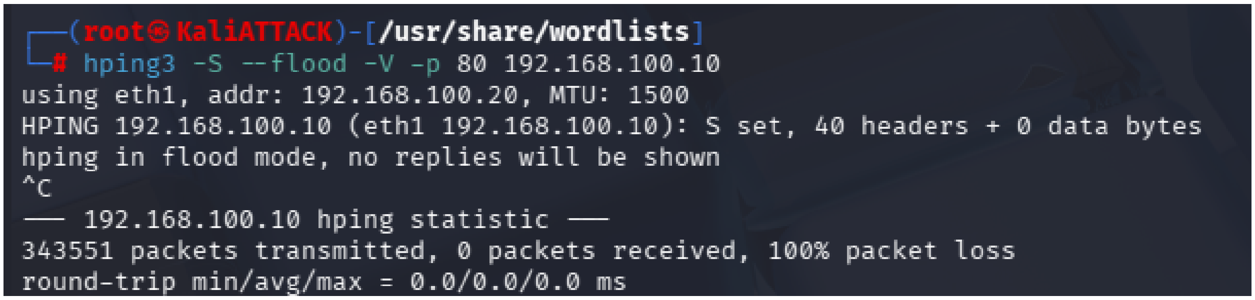
Hydra command output ^

timestamp	agent.name	rule.description	rule.level	rule.id
Apr 21, 2026 @ 01:34:32.068	ubuntuagent1	Suricata: Alert - SURICATA SSH invalid banner	3	86601
Apr 21, 2026 @ 01:34:31.465	kaliagent1	syslog: User missed the password more than one time	10	2502
Apr 21, 2026 @ 01:34:31.463	kaliagent1	syslog: User authentication failure.	5	2501
Apr 21, 2026 @ 01:34:31.461	kaliagent1	Maximum authentication attempts exceeded.	8	5758
Apr 21, 2026 @ 01:34:31.316	UbuntuSIEM	Suricata: Alert - SURICATA Applayer Detect protocol only one dir...	3	86601
Apr 21, 2026 @ 01:34:31.316	UbuntuSIEM	Suricata: Alert - SURICATA Applayer Detect protocol only one dir...	3	86601
Apr 21, 2026 @ 01:34:31.316	UbuntuSIEM	Suricata: Alert - SURICATA Applayer Detect protocol only one dir...	3	86601
Apr 21, 2026 @ 01:34:31.316	UbuntuSIEM	Suricata: Alert - SURICATA Applayer Detect protocol only one dir...	3	86601
Apr 21, 2026 @ 01:34:31.316	UbuntuSIEM	Suricata: Alert - SURICATA Applayer Detect protocol only one dir...	3	86601
Apr 21, 2026 @ 01:34:31.315	UbuntuSIEM	Suricata: Alert - SURICATA Applayer Detect protocol only one dir...	3	86601
Apr 21, 2026 @ 01:34:30.893	ubuntuagent1	Suricata: Alert - SURICATA SSH invalid banner	3	86601
Apr 21, 2026 @ 01:34:30.877	ubuntuagent1	Suricata: Alert - SURICATA SSH invalid banner	3	86601
Apr 21, 2026 @ 01:34:30.873	ubuntuagent1	Suricata: Alert - SURICATA Applayer Detect protocol only one dir...	3	86601
Apr 21, 2026 @ 01:34:30.433	ubuntuagent1	Suricata: Alert - SURICATA SSH invalid banner	3	86601
Apr 21, 2026 @ 01:34:30.422	ubuntuagent1	Suricata: Alert - SURICATA SSH invalid banner	3	86601

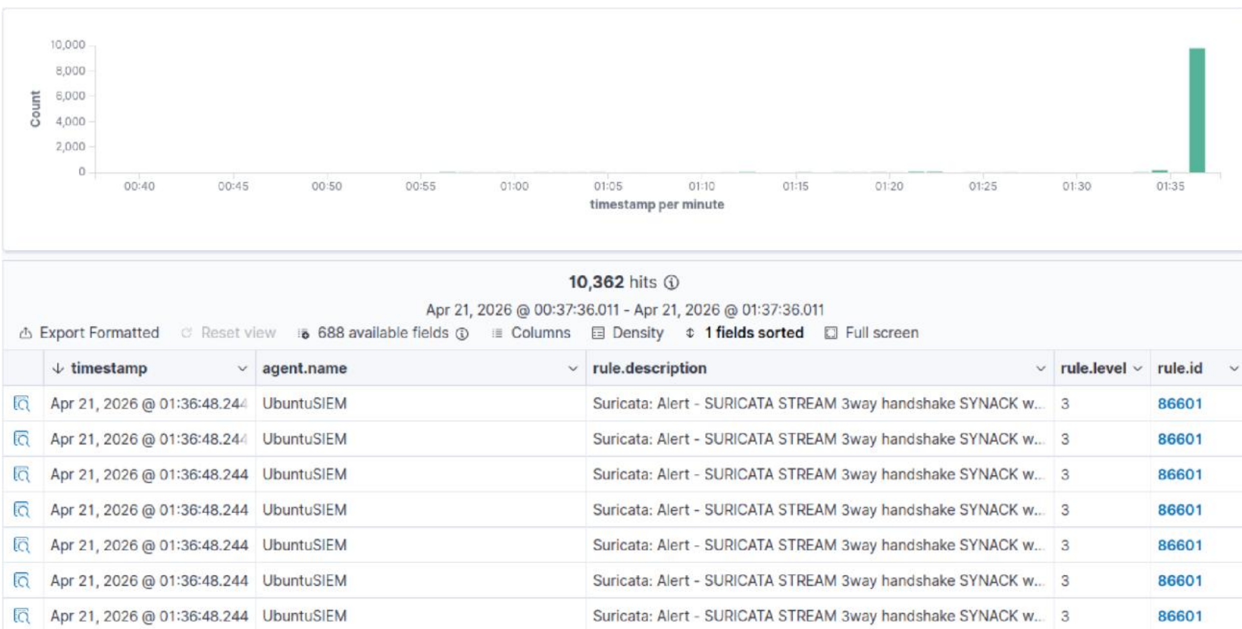
Suricata Alerts captured in Wazuh ^

SYN Flood DOS

Next, we ran a SYN flood DOS attack on the victim machine with the attacker targeting port 80, it sent 343,551 packets to the victim machine. Resulting in just a couple (over 10,000) of alerts to go off ;)



SYN flood targeting port 80 on victim machine ^



Suricata capturing SYN flood alerts ^

ARP Spoofing

Next, we decide to Perform an ARP spoof on the attacker machine to trick it to send network Traffic to the attacker VM instead of hte real gateway (MitM) Attack.

```
┌──(root㉿KaliATTACK)-[/usr/share/wordlists]
└─# sudo arpspoof -i eth1 -t 192.168.100.10 192.168.100.1
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 8:0:27:3f:20:4d
^CCleaning up and re-arping targets...
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 0:e0:4c:68:4b:db
^C8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 0:e0:4c:68:4b:db
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 0:e0:4c:68:4b:db
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 0:e0:4c:68:4b:db
8:0:27:3f:20:4d 8:0:27:83:bd:57 0806 42: arp reply 192.168.100.1 is-at 0:e0:4c:68:4b:db
```

Nikto Web Scan

We then ran a Nikto web scan on the victim Apache server to discover misconfiguration inside of the server. We founce XSS vulnerability and 400 error.

```
┌──(root㉿KaliATTACK)-[/usr/share/wordlists]
└─# nikto -h 192.168.100.10
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.100.10
+ Target Hostname:    192.168.100.10
+ Target Port:        80
+ Start Time:         2026-04-20 19:49:48 (GMT-6)
---------------------------------------------------------------------------
+ Server: Apache/2.4.66 (Debian)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozil
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render t
ing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /: Server may leak inodes via ETags, header found with file /, inode: 29cf, size: 64d2f9c7ff
+ OPTIONS: Allowed HTTP Methods: OPTIONS, HEAD, GET, POST .
^C
```

Nikto Scan Command ^

	Apr 21, 2026 @ 01:50:14.044	kaliagent1	Web server 400 error code.	5	31101
	Apr 21, 2026 @ 01:50:14.042	kaliagent1	Web server 400 error code.	5	31101
	Apr 21, 2026 @ 01:50:14.039	kaliagent1	Multiple web server 400 error codes from same source ip.	10	31151
	Apr 21, 2026 @ 01:50:12.340	kaliagent1	XSS (Cross Site Scripting) attempt.	6	31105
	Apr 21, 2026 @ 01:50:12.292	kaliagent1	Common web attack.	6	31104

Multiple Alerts from the website scan ^

FTP Brute Force

Next, we decided to try and do another brute force attack, this time it was against FTP (port 21) to try and get access to the file server. This was done using Hydra, and below the command picture you can see that Suricata was able to catch these attempts and alerted us right away.

```
┌──(root㉿KaliATTACK)-[/usr/share/wordlists]
└─# hydra -l root -P rockyou.txt 192.168.100.10 ftp -t 4 -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret servic

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-20 19:56:11
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries
[DATA] attacking ftp://192.168.100.10:21/
[ATTEMPT] target 192.168.100.10 - login "root" - pass "123456" - 1 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "12345" - 2 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "123456789" - 3 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "password" - 4 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "iloveyou" - 5 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "princess" - 6 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "1234567" - 7 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "rockyou" - 8 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "12345678" - 9 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "abc123" - 10 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "nicole" - 11 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "daniel" - 12 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "babygirl" - 13 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "monkey" - 14 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "lovely" - 15 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "jessica" - 16 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "654321" - 17 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "michael" - 18 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "ashley" - 19 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "qwerty" - 20 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "111111" - 21 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "iloveu" - 22 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "000000" - 23 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "michelle" - 24 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "tigger" - 25 of 14344399 [child 2] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "sunshine" - 26 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "chocolate" - 27 of 14344399 [child 1] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "password1" - 28 of 14344399 [child 3] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "soccer" - 29 of 14344399 [child 0] (0/0)
[ATTEMPT] target 192.168.100.10 - login "root" - pass "anthony" - 30 of 14344399 [child 2] (0/0)
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.
```

FTP Brute Force Command ^

	Apr 21, 2026 @ 01:56:36.282	kaliagent1	unix_chkpwd: Password check failed.	5	5557
	Apr 21, 2026 @ 01:56:36.282	kaliagent1	unix_chkpwd: Password check failed.	5	5557
	Apr 21, 2026 @ 01:56:36.278	kaliagent1	unix_chkpwd: Password check failed.	5	5557
	Apr 21, 2026 @ 01:56:34.260	kaliagent1	unix_chkpwd: Password check failed.	5	5557
	Apr 21, 2026 @ 01:56:34.260	kaliagent1	PAM: User login failed.	5	5503
	Apr 21, 2026 @ 01:56:32.287	kaliagent1	PAM: Multiple failed logins in a small period of time.	10	5551

SIEM Alerts ^

Resources Used

Open Information Security Foundation. (n.d.). *Suricata user guide*. Suricata documentation. https://docs.suricata.io/

Wazuh, Inc. (n.d.). *User manual*. Wazuh documentation. https://documentation.wazuh.com/current/user-manual/index.html

The Ubuntu manpages team. (n.d.). *man - an interface to the on-line reference manuals*. Ubuntu manpages. https://manpages.ubuntu.com/manpages/trusty/man1/man.1.html

Kali Linux. (n.d.). *Kali docs*. Kali Linux documentation. https://www.kali.org/docs/